

FREE PROMPT PACK · CLAUDE FABLE 5.1 · SECOND EDITION

Build Your Own *Jarvis*

Eight copy-paste prompts that turn a folder of your own notes into a talking, seeing, self-verifying AI second brain. Rebuilt for Claude Fable 5.1 — where one prompt now finishes what used to take three.

How to use this pack

What you need: Claude Code installed and logged in, Google Chrome (the microphone and speech APIs need it), a folder of markdown notes, and an Anthropic API key from console.anthropic.com. Five dollars of credit is more than enough to run this whole pack.

Set up in one line. Open a terminal, make an empty folder, and start Claude Code inside it: `mkdir my-jarvis && cd my-jarvis && claude`

1 Set the model first, before anything else.

Type `/model claude-fable-5-1` in Claude Code. Every prompt in this pack assumes you are on Fable 5.1. On an older model the longer prompts will still work, but you will have to babysit them in smaller pieces, exactly the way the first edition of this pack did.

2 Paste each prompt whole, unedited, the first time.

Each one is a complete brief: what to build, how to structure it, and how to prove it works. Run it as written before you start customizing. The endings matter as much as the beginnings.

3 Never paste your API key into a chat window.

Prompt 01 creates a config file with a placeholder. You type the real key into that file yourself, in your own editor. Treat anything pasted into any chat as exposed. This is the one rule with no exceptions.

4 Let it finish. Resist the urge to interrupt.

These prompts run long on purpose. Fable 5.1 is built for exactly this: it will write a file, run it, read the error, fix the cause, and try again. Interrupting mid-loop is now the most common way people get a worse result.

5 Direct it with feelings, not specifications.

After the first build, say things like "the voice is too slow to answer," "this feels cluttered," "it talks too much." Describing the problem beats prescribing the fix, because it can find the cause and you can only guess at it.

What changed since the last pack

The first edition of this pack shipped in July, built for Claude Opus 4.8. It had six prompts and one instruction repeated throughout: run one, test it, then run the next. That instruction existed because the model of the day lost the thread on long builds. Here is what is genuinely different now, and what is not.

	FIRST EDITION · OPUS 4.8	THIS EDITION · FABLE 5.1
Prompt size	Six small, carefully fenced steps. The galaxy and the brain were two separate prompts because asking for both at once tended to produce half of each.	Eight prompts, each far larger. Prompt 01 alone replaces the old prompts 1 and 2 , and finishes with a working server you can talk to.
Working style	You supervised every step and tested between each one.	It builds, runs, reads its own errors, and fixes the cause rather than the symptom before it hands anything back.
Verification	Manual. You clicked around and hoped.	Prompt 07 has it build a preflight harness that tests every live chain and prints a pass or fail verdict. This is the single biggest reliability upgrade in the pack.
Cost of a long build	Long agentic sessions were the expensive kind.	Roughly 25% cheaper on typical work and up to about 45% cheaper on heavy tool-use sessions, because cached reads dropped 75%. Long sessions are now the cheap ones.
What it can hold	You reminded it what it had already built.	A 1M token context window. It keeps the whole project in its head, so later prompts can say "the server you built in prompt 01" and it knows.
New ground	Text and voice only.	Prompt 06 gives it sight . Prompt 08 lets it swap its own brain by voice, mid-conversation.

What did not change. You still supervise. It still needs a real API key that you pay for. The voice still only works properly in Chrome. And it will still occasionally build something confidently wrong, which is exactly why prompt 07 exists and why it is not optional. Anyone telling you this part is solved has not shipped anything.

The Galaxy and the Brain

Best for: the first paste. This one prompt replaces both opening prompts of the previous pack.

Build me a 3D knowledge galaxy from my markdown notes, and give it a brain I can talk to. Do the whole thing in one pass, then start it and verify it before you tell me it's done.

MY NOTES: [PASTE THE FULL PATH TO YOUR NOTES FOLDER – or delete this line and write "I have no notes, so generate 30 realistic sample notes about a small business into ./notes first"]

THE INDEXER – build.py, Python 3, standard library only, no pip installs. Scan every .md file and write viewer/graph-data.js containing `const GRAPH = {nodes: [...], links: [...]}`. Each node gets: a label from the filename, its parent folder as a group, and roughly a 700-character excerpt. Link two notes when one mentions the other's title or they share `[[wikilinks]]`. CRITICAL: give every node a numeric id equal to its index in the nodes array. Later prompts depend on looking nodes up by index, and if you use string ids here I will have to rebuild this.

THE VIEWER – viewer/index.html, single page, using 3d-force-graph from a CDN. No npm, no build step, no framework. Make it cinematic: black space, a starfield, nodes glowing and colour-coded by group, slow idle rotation. Clicking a node flies the camera to it, lights its neighbours, and opens a side panel with that note's excerpt.

THE SERVER – server.py, Python standard library, port 4700, serving ONLY the viewer/ folder. Create config.json in the PROJECT ROOT, never inside viewer/, containing `{"api_key": "PUT-YOUR-KEY-HERE", "model": "claude-fable-5-1"}`. Do not ask me for my key and do not put it anywhere the browser can reach.

THE BRAIN – add POST /chat to server.py. Score every note against my question by keyword overlap, weighting title matches higher, take the top 6, and call the Anthropic Messages API with the model from config.json. System prompt: answer ONLY from these notes, in two or three sentences, and say so plainly when the notes do not cover it. Return `{"answer": "...", "nodes": [array indexes of the notes you used]}`. Keep a short conversation history server-side so follow-up questions work. Add a clean input bar at the bottom of the viewer that shows the answer on screen.

FINISH PROPERLY – start the server, confirm the page loads and the graph renders, and confirm /chat returns a well-formed response with a placeholder key error rather than a crash. Then tell me exactly what to paste into config.json, and give me three example questions based on my actual note titles.

Make it yours: if you would rather not buy an API key, add one line: "instead of the Anthropic API, make /chat shell out to `claude -p` so it runs on my Claude Code subscription." Slower, but free.

The Voice, both directions

Best for: the moment it stops being a website and starts being an assistant.

Give it a voice, both directions, using only what the browser already has. No paid speech services.

SPEAKING – every answer is spoken aloud with the Web Speech API (`speechSynthesis`). Prefer a British English voice when the system has one. Browsers block audio until the user has interacted with the page, so make sure the first spoken line reliably fires after my first click, not before.

LISTENING – add a microphone button using `webkitSpeechRecognition`. I click it, I speak, the transcript goes through the same /chat flow as typing.

THE PART EVERYONE GETS WRONG – speech recognition finalizes a phrase when I pause, and I pause in the middle of sentences. So do NOT dispatch on the first final result. Buffer it, and put the wait in a single named constant at the top of the file called `FINISH_MS`, set to 900. If more speech arrives before that timer fires, append it and restart the timer. The pause that actually ends my thought sends the whole combined sentence. Short interrupt words like "stop" and "wait" must bypass the buffer entirely and fire instantly.

STATUS – a small status line that shows listening, thinking, and speaking states, so I always know which of us the machine thinks is talking.

VERIFY – then give me a test plan: exactly what to click, what to say, and what I should see and hear at each step, including one deliberate mid-sentence pause to prove the finish window works.

Make it yours: `FINISH_MS` is the single most personal number in this build. Too low and it cuts you off mid-thought. Too high and it feels sluggish. Start at 900, then tell Claude to raise or lower it after a day of real use. Do not guess at it in one sitting.

Make it prove where the answer came from

Best for: the ten seconds of this build that make people lean toward the screen.

When it answers from a note, I want the galaxy to prove it.

/chat already returns a nodes array of the indexes it used. Use it:

- Fly the camera to the top source node, light it up along with its direct neighbours, and open its side panel, so I can see exactly which note the answer came from while it is still speaking.
- If the answer drew on four or more notes, do not fly anywhere. Light the whole cluster instead. Flying to one arbitrary node out of six is a lie about where the answer came from.
- Never read the note aloud. It is on screen. The spoken answer stays short and the screen carries the detail.
- Small talk must not drag the camera around. If I say "good morning" or make a joke, the graph holds still. Only actual questions about my notes move the camera.

That last rule is the one that separates a demo from a toy, so implement it deliberately rather than as an afterthought: decide whether the question was about my notes before you decide whether to move the camera.

Then show me: give me one question that should fly to a single note, one that should light a cluster, and one that should not move the camera at all.

Make it yours: this is the shot worth recording. Ask something whose answer lives in one very specific note, and film the camera dive. It reads on video far better than any feature list.

The Butler

Best for: turning a question box into something people actually want to talk to.

Give it a personality. Rewrite the /chat system prompt so the assistant is a dry, impeccably polite British butler with a razor wit. Rules:

- Address me as "sir", occasionally, not in every sentence. Over-using it is the difference between charming and grating.
- Answer in ONE witty sentence plus the facts. Never recite the note back to me, it is already on my screen.
- One genuinely funny line beats three bland ones. If nothing funny is available, be brief instead of forcing it.
- Handle small talk without dragging the camera around the graph.
- When my notes genuinely do not cover something, say so plainly and with a little dignity. Never invent a source, never pad, and never pretend a related note is the answer.

BOOT GREETING – when the page loads, it greets me with the real indexed note count, for example "Good evening, sir. 485 notes indexed, all present and accounted for." Read the actual number from the graph data, do not hardcode it. Make the time of day correct too.

Put the entire persona in ONE clearly commented block at the top of server.py, so I can rewrite the character later without hunting through the file.

Then give me five things to say to it that will show off the personality, including one question it definitely cannot answer from my notes.

Make it yours: the butler is just the default. Swap the persona block for a terse mission-control operator, a sarcastic friend, or a patient teacher. Keeping it in one block is what makes that a thirty-second change.

Total Recall

Best for: the point where the brain starts getting bigger without you opening an editor.

Let me grow the brain by voice.

- When I say or type anything starting with "remember that", POST it to a new /remember endpoint that writes a real markdown file into a captures/ folder inside my notes directory. Give it a sensible title derived from the first few words, and stamp today's date inside the file.
- Add the new node to the running galaxy LIVE, without a page reload. Born at the position of its most related existing node, with a brief glow pulse, then fly the camera to it.
- Link it into the graph the same way build.py would, so it is a real citizen of the galaxy and can be used as a source in future answers immediately.
- Confirm out loud in one line, in character.

Two things to get right, because they are the ones that bite later:

1. Writing a file is not the same as indexing it. Make sure the new note is searchable by /chat in the very next question, without re-running build.py.
2. Never let a capture silently fail. If the write fails for any reason, say so out loud. A second brain that quietly forgets things is worse than no second brain.

Then have me test it: I will say "remember that the finish window should be 900 milliseconds", and you show me the new star being born, then I will immediately ask a question that should retrieve it.

Make it yours: add trigger phrases that match how you actually talk. "note to self", "don't let me forget", "log this". The capture only works if the phrase is one you would say without thinking about it.

The Eyes

Best for: the first time it answers a question about something you never described to it.

Give it sight. I want to be able to point at my own screen and ask "what do you think of this?"

- Add a screen button that starts a `getDisplayMedia` screen share and holds the stream. While a share is live, show a small, unmistakable indicator so I always know it can see. This is not a feature to be subtle about.
- When I ask a question while sharing, grab a single frame from the stream to a canvas, encode it as JPEG, and send it to a new `POST /see` endpoint alongside my question.
- `/see` calls the Anthropic Messages API with the image plus my question, using the same model from `config.json`. Fable 5.1 reads dense screenshots well, so tell it to answer specifically about what is actually on screen and to say plainly when the screenshot is too small or blurry to judge, rather than guessing.
- The answer comes back through the same voice and the same on-screen path as every other answer.

BUILD IT SO IT CANNOT LIE TO ME:

- The frame must be captured at the moment I ask, not cached from when the share started.
- If the share has ended, it must say the share ended. It must never answer a screen question from memory of an earlier frame.
- Encode as JPEG and send it as `image/jpeg`. Match the media type exactly to what you actually encode, or the endpoint will fail in a way that looks like the whole feature is broken when it is really one wrong string.

Verify it end to end in front of me: start a share, ask "what am I looking at", and show me the real answer.

Make it yours: screen sharing is far more useful day to day than a webcam, and much easier to trust. Point it at a spreadsheet, an error message, or a design and ask what it would change.

The Preflight

Best for: the difference between a weekend demo and something you actually rely on.

Build me a preflight harness. This is the most important prompt in this pack, so take your time with it.

Write preflight.py that executes every LIVE chain in this project end to end and prints a verdict. Not unit tests. Not mocks. Real calls against the running system, because the failures that hurt are the ones where every unit test passes and a real chain is dead.

It must check, in order, and print a tick or a cross for each:

- The server is up and serving the viewer.
- The graph data loads and the node count is greater than zero.
- /chat returns a well-formed answer to a real question, with a nodes array.
- The API key in config.json is actually valid, by making one real minimal call.
- The model named in config.json is one the key can actually reach.
- /remember writes a real file and the new note is retrievable by /chat immediately afterwards.
- The files the browser is actually being SERVED match the files on disk. A stale served file is the single most common cause of "I fixed it but nothing changed".
- config.json is not reachable from the browser. This one must fail loudly.

Finish with a single summary line: N pass, N fail, N warn. Exit non-zero on any failure.

Then run it and show me the output. If anything fails, fix the cause and run it again until it is clean. Do not describe what it would say. Run it.

From now on, whenever you tell me a change is done, run preflight.py first and paste the summary line. "Done" means preflight passed, not that the code looks right.

Make it yours: add a check every time something breaks on you. A harness that grows one check per real incident becomes the most valuable file in the project within a month. Mine has 43 checks and every one of them was earned.

The Brain Swap

Best for: changing which model is thinking, out loud, mid-conversation.

Let me change its brain by voice, while it is running.

- Add POST /model that swaps the model at runtime. Support spoken names: "switch to Fable 5.1", "try on Haiku", "go back to your normal brain".
- Map spoken names to real model ids: "fable 5.1" becomes claude-fable-5-1, "haiku" becomes the newest Haiku you know about. Keep the mapping in ONE dictionary near the top of the file.
- Swaps are runtime only. A restart always returns to the model in config.json, so I can never strand myself on a brain I did not mean to keep.
- Show the current brain on screen as a small chip, and have it introduce itself in one dry line when it changes.

NOW THE RULE THAT MAKES THIS SAFE, AND I MEAN THIS ONE:

Keep an explicit set of the real model ids you know exist. When I name a family plus a version, build the candidate id and check it against that set. If it is not there, REFUSE and tell me what you do have. Never fall back to the nearest match.

The failure this prevents is specific and nasty: I say "opus 5", a loose matcher sees the word "opus", throws away the version, loads Opus 4.8, and cheerfully announces it has done what I asked. I then spend an hour testing the wrong model. An honest error is worth more than a helpful guess, every single time.

Test it in front of me: swap to a real model, swap to a version that does not exist and show me the refusal, then restart the server and show me it came back on the config model.

Make it yours: the refusal rule is the transferable lesson here. Anywhere you build maps something a human said onto something exact, make the near-miss fail loudly instead of guessing. Silent wrong answers cost more than loud errors.

Five things I learned building this

1 **Longer prompts now beat more prompts.**

The instinct from last year was to break everything into small safe steps. On Fable 5.1 that actively costs you, because every handoff is a chance to lose context it was perfectly capable of holding.

2 **Make it verify, or you are just hoping.**

Prompt 07 is the one people skip and the one that decides whether this thing survives past the weekend. Build the harness before you build more features.

3 **Tune from numbers, never from memory.**

When the voice feels wrong, print the actual timings before changing anything. Every value worth arguing about should be a single named constant at the top of its file.

4 **An honest error beats a helpful guess.**

Every hour I have lost on this project was lost to something that failed quietly and told me it had succeeded. Build the loud failure in from the start.

5 **Personality is not decoration.**

Nobody uses the version with no character. The butler is the reason it gets opened on a Tuesday, and that is the whole difference between a project and a habit.

Want the full system, not just the prompts?

This pack gets you a working second brain. Inside the paid AI Workshop community you get the complete builds I actually run: the phone line, the screen watcher, the automation registry, and the skill installs behind all of it, plus the playbooks for turning these into paid client work.

Free community: skool.com/aiworkshop-lite

Full workshop: skool.com/aiworkshop

YouTube: [Zubair Tratzada | AI Workshop](#)